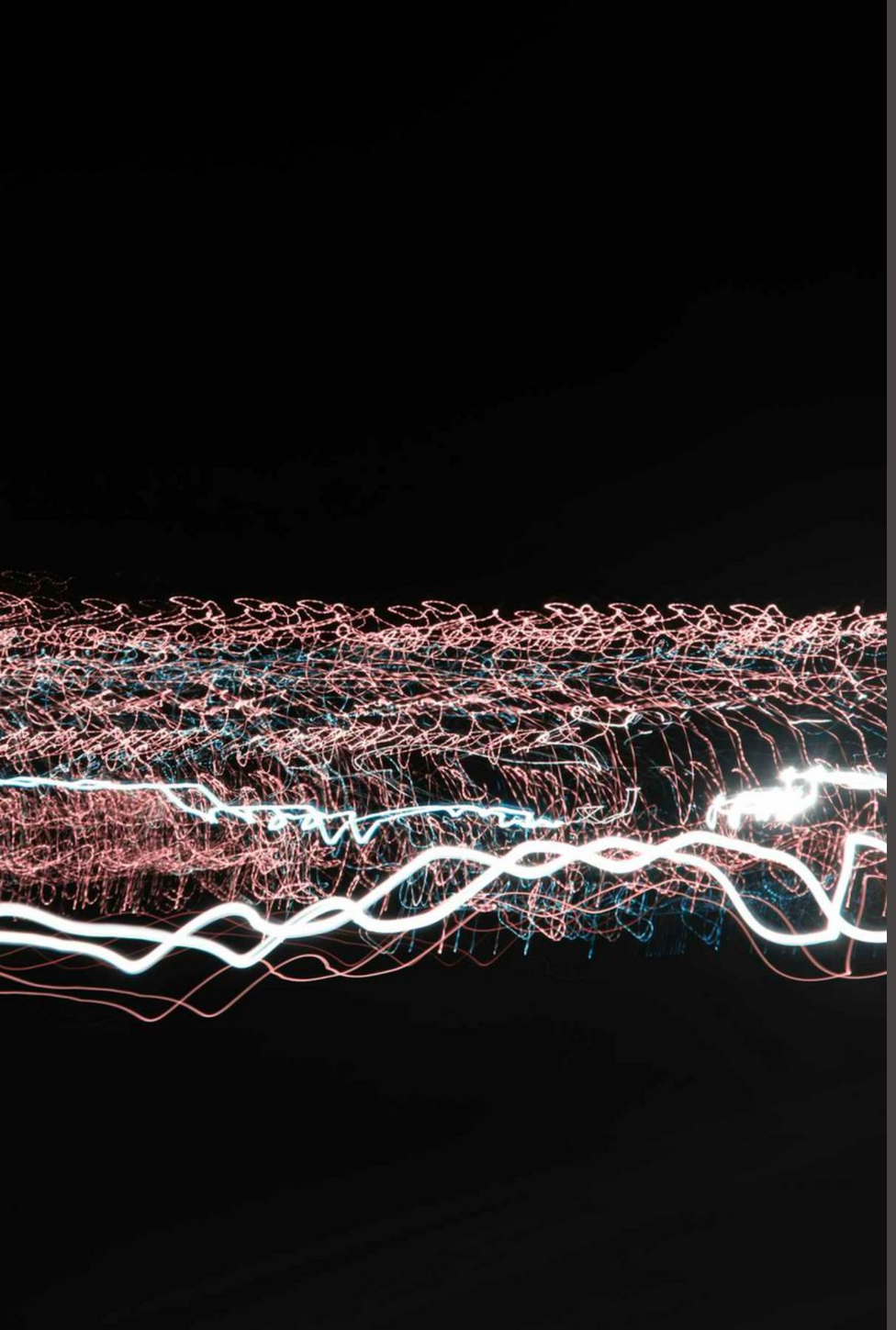


Graph Theory & Adjacency Matrices

Subject: Data Structures
Seminar Presentation by Group 3 Members



Introduction to Graphs

A **graph** is a fundamental data structure consisting of **vertices (nodes)** connected by **edges**. Graphs model relationships and connections, making them essential in computer science.

Directed vs Undirected

Edges may have direction (arrows) or be bidirectional connections between nodes.

Weighted vs Unweighted

Edges may carry numerical values representing cost, distance, or capacity.

Graphs power applications across networking infrastructure, social media platforms, GPS navigation systems, recommendation engines, and countless other real-world systems.

Graph Representation Methods

Computer scientists have developed several methods to represent graphs in memory, each with distinct trade-offs for different use cases.

1

Adjacency Matrix

A 2D array storing connections between vertices. **Our focus today.**

2

Adjacency List

An array of lists where each vertex stores its neighbours, efficient for sparse graphs.

3

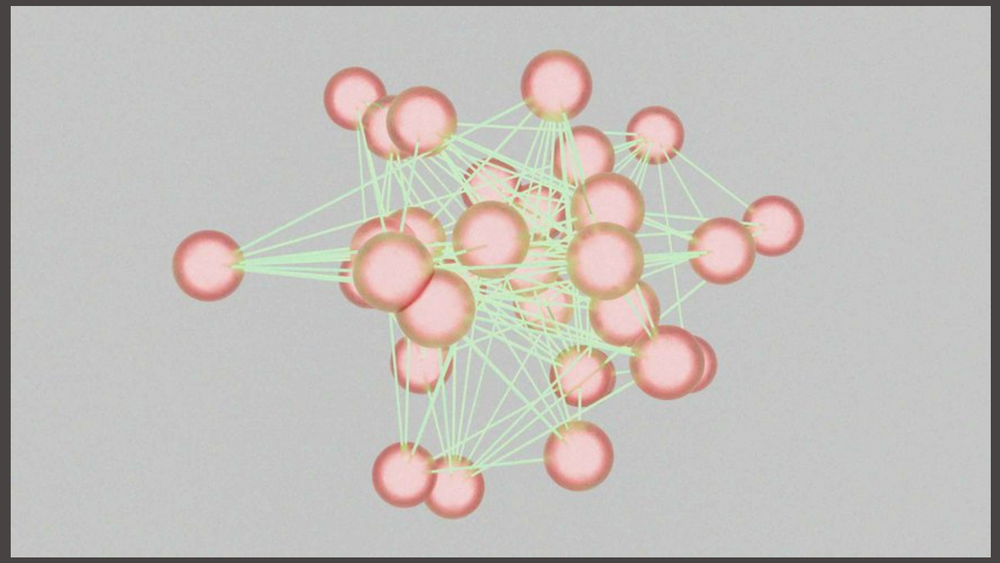
Incidence Matrix

A matrix relating vertices to edges, useful for specific graph algorithms.

In this presentation, we focus only on the Adjacency Matrix representation.

Adjacency Matrix: Definition & Concept

An **adjacency matrix** is a 2D array that represents connections between vertices in a graph. For a graph with **n vertices**, the matrix dimensions are **n × n**.



How It Works

- Each row and column represents a vertex
- Matrix entry [i][j] indicates connection status
- **1** means an edge exists between vertices i and j
- **0** means no edge exists

Weighted Graphs

For weighted graphs, store the **edge weight** instead of 1, with 0 or infinity representing no connection.

Example of Adjacency Matrix

Consider a directed graph with three vertices: A, B, and C, with edges A→B, B→C, and A→C.

Corresponding Adjacency Matrix

	A	B	C
A	0	1	1
B	0	0	1
C	0	0	0

Row A shows connections from vertex A: edges to both B and C are marked with 1. Row B shows only an edge to C. Row C has no outgoing edges.

C Code Example: Creating an Adjacency Matrix

Here's how to implement an adjacency matrix in C, from declaration through user input to display.

```
#include <stdio.h>
#define V 3

int main() {
    int graph[V][V] = {
        {0, 1, 1},  // A → B, A → C
        {0, 0, 1},  // B → C
        {0, 0, 0}   // C → none
    };

    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            printf("%d ", graph[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```



Advantages of Adjacency Matrices

Simplicity

Straightforward to implement using basic 2D arrays. Intuitive structure makes debugging easier for beginners.

Constant-Time Edge Lookup

Checking whether an edge exists takes **O(1)** time—just access adj[i][j] directly.

Ideal for Dense Graphs

Performs efficiently when graphs have many edges relative to vertices, maximising space utilisation.

Disadvantages of Adjacency Matrices

High Space Complexity

Requires **O(n²)** space regardless of edge count. Wasteful for sparse graphs with few connections.

Poor for Sparse Graphs

When a graph has few edges, most matrix entries remain 0, resulting in significant wasted memory.

Costly Vertex Operations

Adding or removing vertices requires reallocating and copying the entire matrix—an expensive operation.

For sparse graphs where edges are much fewer than n^2, adjacency lists typically provide better space efficiency.

Applications in Graph Algorithms

Adjacency matrices serve as the foundation for numerous classical graph algorithms and computational techniques.

Shortest Path Algorithms

The **Floyd–Warshall algorithm** finds shortest paths between all pairs of vertices, leveraging the matrix structure for efficient computation.

Graph Traversal

Depth-First Search (DFS) and **Breadth-First Search (BFS)** can use adjacency matrices to explore vertices and discover graph properties.

Additional Applications

- Network analysis and connectivity checking in telecommunications
- Social network analysis for relationship mapping
- Circuit design and electronic network modelling



Conclusion

The adjacency matrix provides a simple, array-based representation of graphs that balances ease of implementation with powerful computational capabilities.

THANK YOU